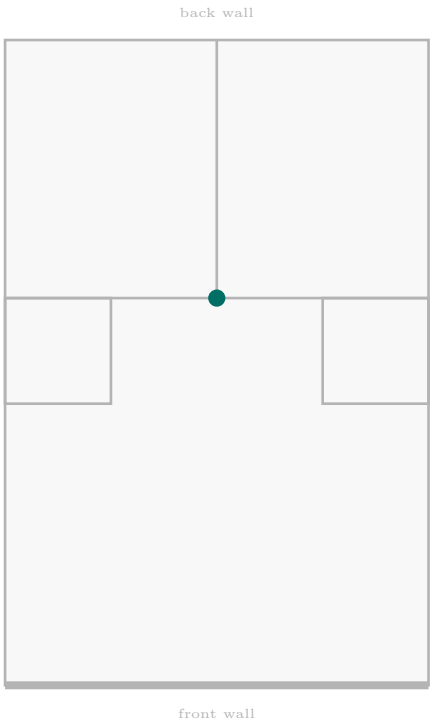
Squash Analysis System

Computer Vision · Sports Analytics

Date	March 24, 2026
Type	Progress Report



Feature Status

Player tracking	Complete
Heatmaps & stats	Complete
Zone breakdown	Complete
Ball tracking	In progress
Rally segmentation	In progress
Web interface	Not started
Shot classification	Not started

Contents

1 Project Overview **3**

2 Pipeline Architecture **3**

3 Completed Work **4**
- 3.1 Court Calibration . . . 4
- 3.2 Player Tracking . . . 4
- 3.3 Player Heatmaps . . . 6
- 3.4 Zone Breakdown . . . 7
- 3.5 Movement Statistics . . . 8
- 3.6 Testing . . . 9

4 In-Progress Work **10**
- 4.1 Ball Detection . . . 10
- 4.2 Ball Tracking (Kalman Filter) . . . 10
- 4.3 TrackNetV4 (Experimental) . . . 11
- 4.4 Rally Segmentation . . . 12

5 Known Issues & Blockers **14**

6 Remaining Planned Work **14**

7 Technical Stack **15**

8 Summary **15**

Project Overview

This project is an open-source, locally-run squash match analysis tool. The goal is to replicate the core features of commercial products (e.g. Rally Vision, SmartSquash) at zero per-match cost, with no video uploaded to third-party servers.

The system processes a single-camera match video and produces:

- Player movement heatmaps and court-space trajectories
- Distance, speed, and T-position statistics
- Zone-based court coverage breakdown
- Ball trajectory visualisation and rally segmentation

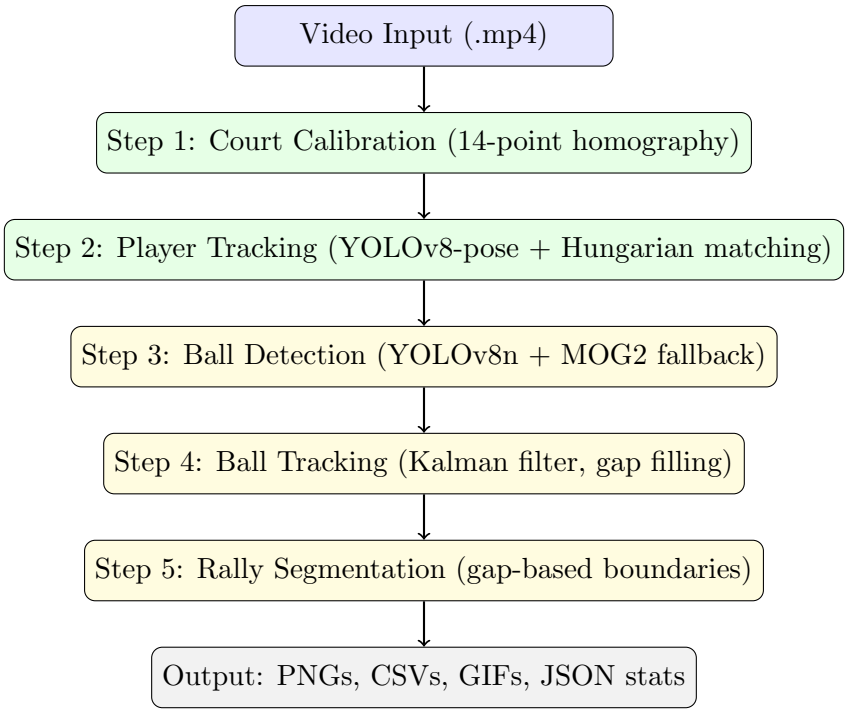
Development Timeline

The project is organised into a 5-week plan. At the time of writing, roughly the first two weeks of work have been completed.

Week	Theme	Status
1	Player tracking, court calibration, heatmaps	**Complete**
2	Movement metrics, YOLO pose tracker, zone breakdown	**Mostly complete**
3	Streamlit web UI, in-browser calibration, PDF reports	**Not started**
4	Ball tracking, rally segmentation, shot classification	**In progress (blocked)**
5	Shot outcomes, highlights, Docker deployment	**Not started**

Pipeline Architecture

The analysis pipeline runs as a series of sequential steps, each writing intermediate results to disk so that expensive stages can be skipped on re-runs.



Steps 1–2 are reliable and run in production. Steps 3–5 are implemented but blocked by ball detection accuracy.

Completed Work

Court Calibration

A 14-point click calibration UI allows the user to mark known court landmarks on the first frame of a video. These are mapped to real-world court coordinates (in metres) using OpenCV's `findHomography`. The resulting 3×3 matrix is cached to disk (`assets/homography.npy`) and reused on subsequent runs unless `--calibrate` is passed.

This homography is used throughout the pipeline to convert pixel coordinates to court-space positions, enabling all distance and speed calculations to be in metres rather than pixels.

Player Tracking

Current approach (Day 12, default): YOLOv8n-pose runs full-frame inference on every video frame. The two highest-confidence person detections are extracted. A cost matrix is built from Euclidean distance in pixel space, and the Hungarian algorithm (via `scipy.optimize.linear_sum_assignment`) assigns detections to Player 1 and Player 2, maintaining consistent identities across frames.

Ground positions are derived from pose keypoints using a three-tier fallback:

1. Ankle midpoint (COCO keypoints 15 & 16) — preferred
2. Knee midpoint (keypoints 13 & 14) — if ankles not visible
3. Hip midpoint (keypoints 11 & 12) — final fallback

Legacy approach: A MediaPipe-based tracker crops the frame into top and bottom halves before running pose estimation. This approach is retained for comparison but is prone to identity swaps when players are close together, and the hip-only fallback introduces a systematic ∼1 m upward bias in the Y-position estimate.

Known limitation: Player initialisation (assigning which detection is P1 vs P2) relies on vertical position in the first frame. If both players appear on the same half of the court initially, assignment is incorrect and persists for the entire clip.

Figure 1 shows the tracked player positions overlaid directly on the video frame, and Figure 2 shows the same trajectories projected into court space via the homography.

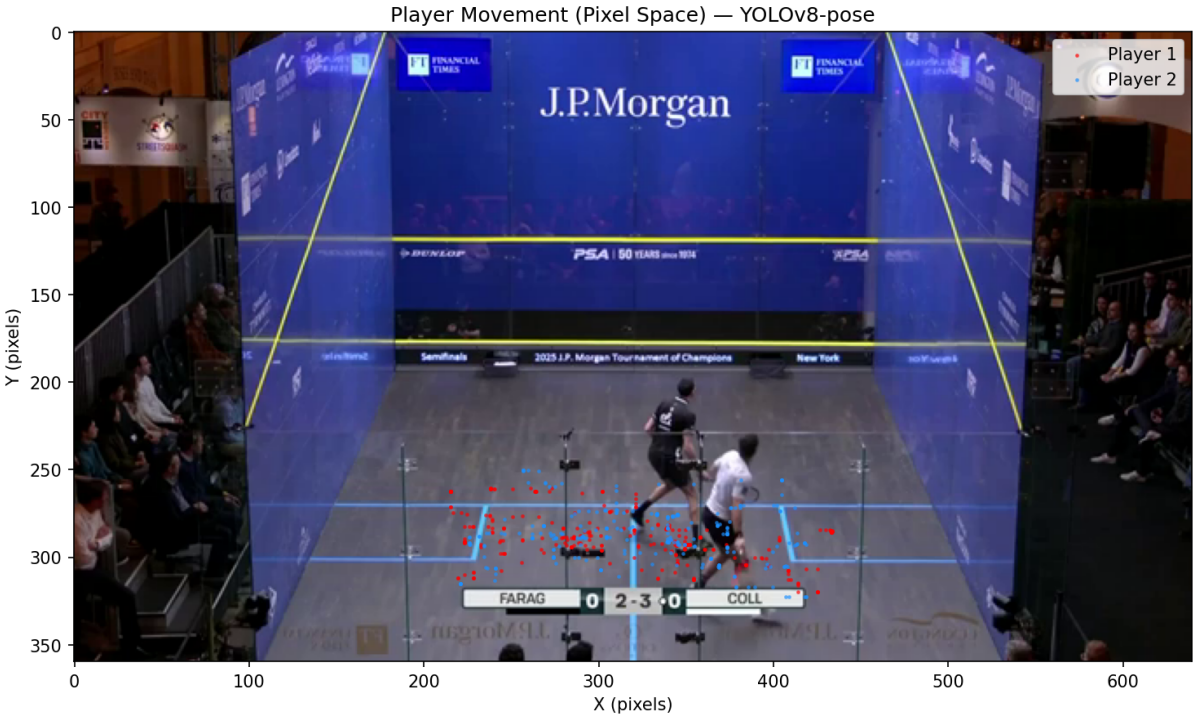


Figure 1: Player movement in pixel space overlaid on the match video. Red dots = Player 1, blue dots = Player 2. The yellow lines show the calibrated court boundary used for the homography.

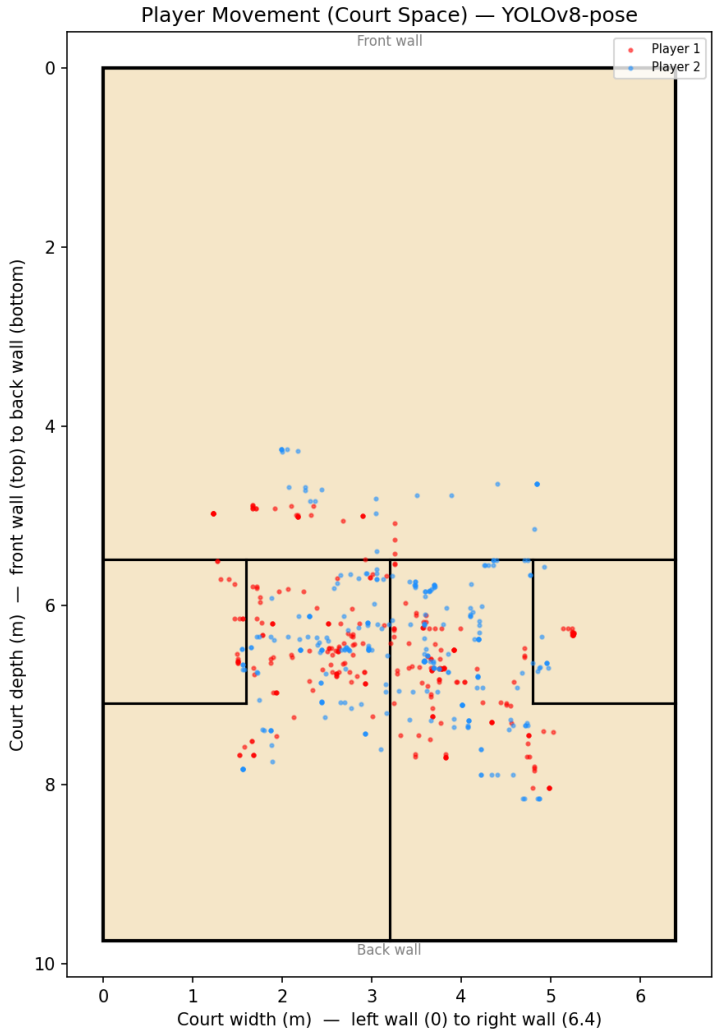
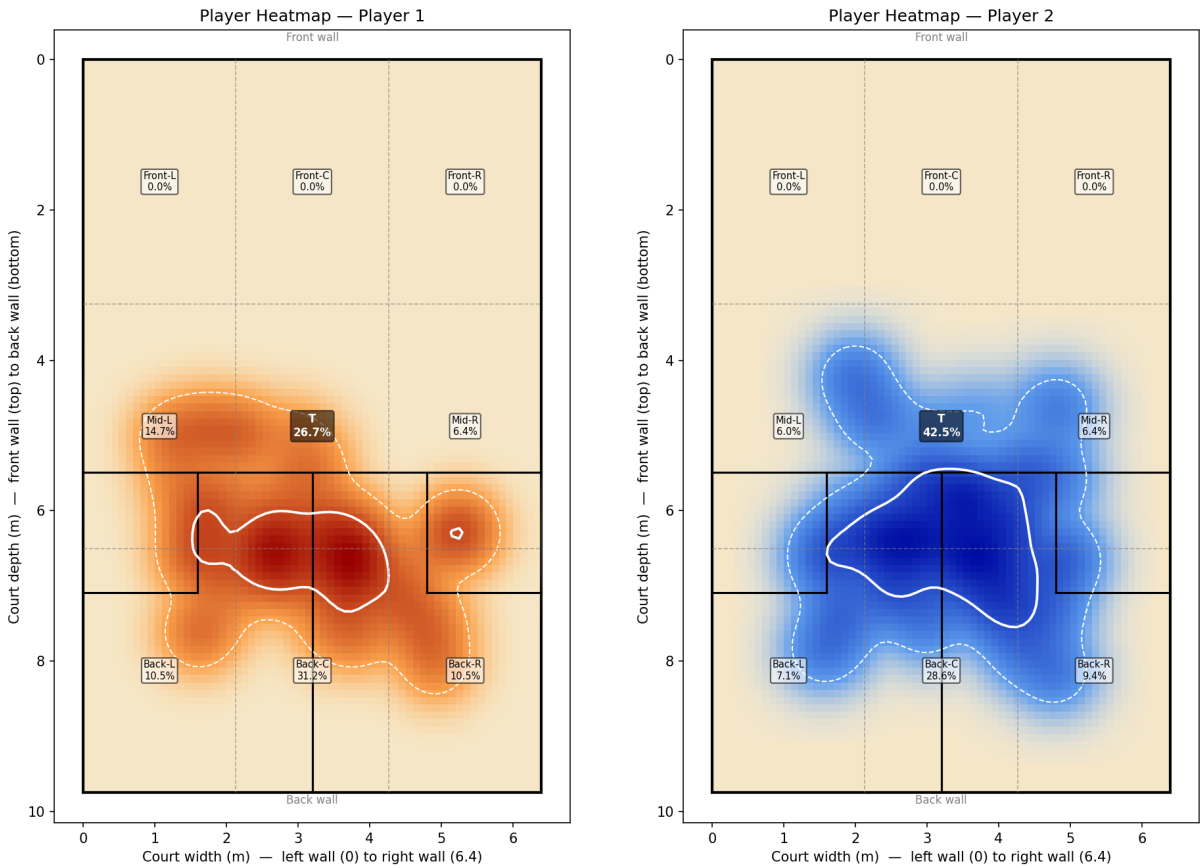


Figure 2: Player trajectories projected into court space (metres). Each dot is one frame. The court diagram is drawn to scale: 9.75 m deep × 6.4 m wide.

Player Heatmaps

Gaussian kernel density estimation is applied to the court-space position sequence for each player, producing a smooth heatmap that shows where each player spends the most time. Figure 3 shows the resulting heatmaps side by side.



(a) Player 1 — concentrated around back-centre and the T.

(b) Player 2 — tighter concentration around the T zone.

Figure 3: Gaussian heatmaps for both players. Warmer colours indicate more time spent in that region. The T junction is marked with a white label.

Zone Breakdown

The court is divided into a 3×3 grid (Front/Mid/Back $\times$ Left/Centre/Right), matching standard squash tactical analysis zones. The fraction of time each player spends in each zone is computed and displayed as a percentage. Figure 4 shows the zone breakdown for the analysed clip.

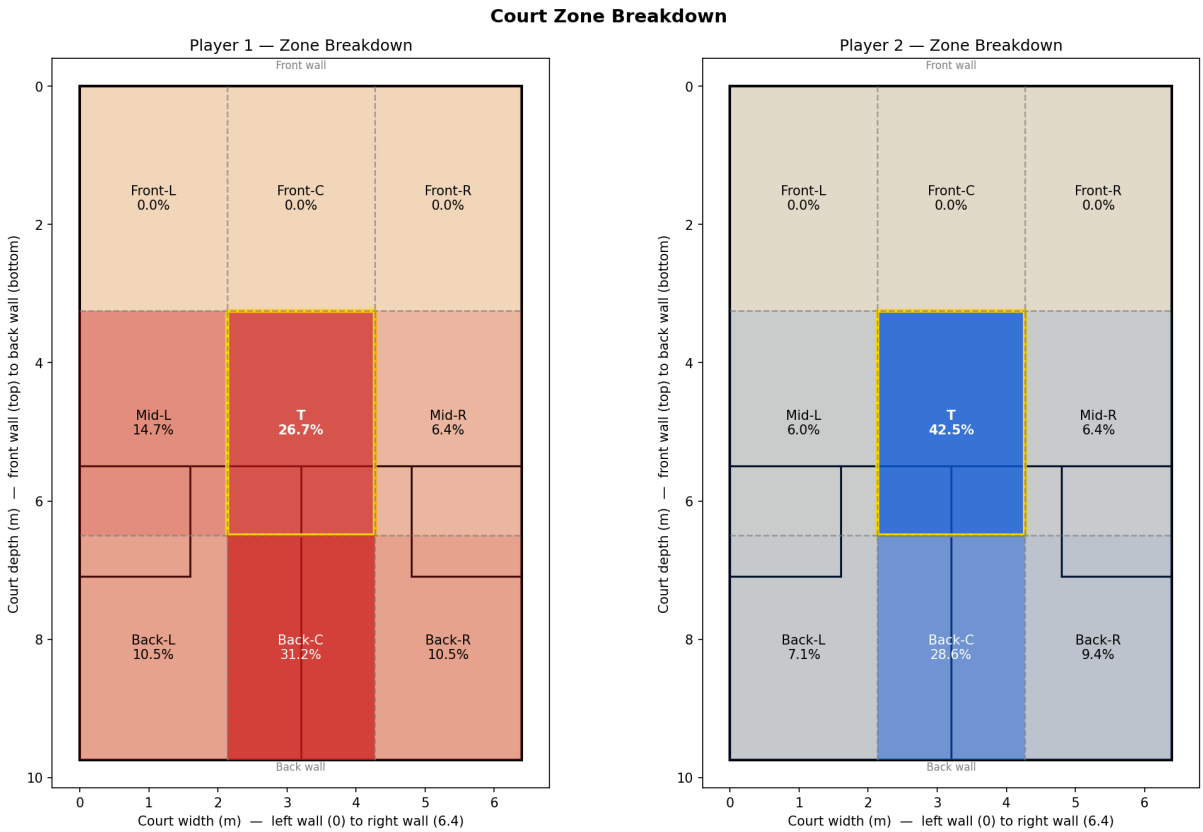


Figure 4: Zone-by-zone court coverage for each player. The T zone (centre of mid-court) is highlighted in gold. Player 1 spent 26.7% of the clip on the T; Player 2 spent 42.5%.

Movement Statistics

The following per-player statistics are computed from court-space position sequences:

Metric	Method	Unit
Total distance	Sum of frame-to-frame Euclidean distances	m
Average speed	Total distance / clip duration	m/s
Peak speed	95th percentile of instantaneous speeds	m/s
T-position time	% of frames within 1.25 m of T junction	%
Front/back split	Fraction of positions with $y < 5.5$ m (mid-court)	%

Figure 5 shows three time-series analytics plots: instantaneous speed, distance from the T, and court depth over the duration of the clip. Figure 6 shows the marginal position distributions for both players.

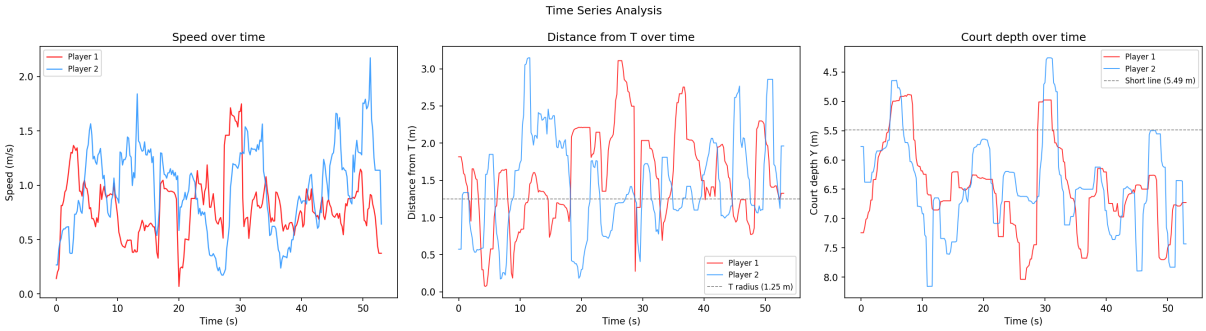


Figure 5: Time-series analytics. *Left:* instantaneous speed (m/s). *Centre:* distance from the T junction (m); the grey dashed line marks the 1.25 m T-proximity threshold. *Right:* court depth over time (m); the horizontal line at 5.49 m marks the short line.

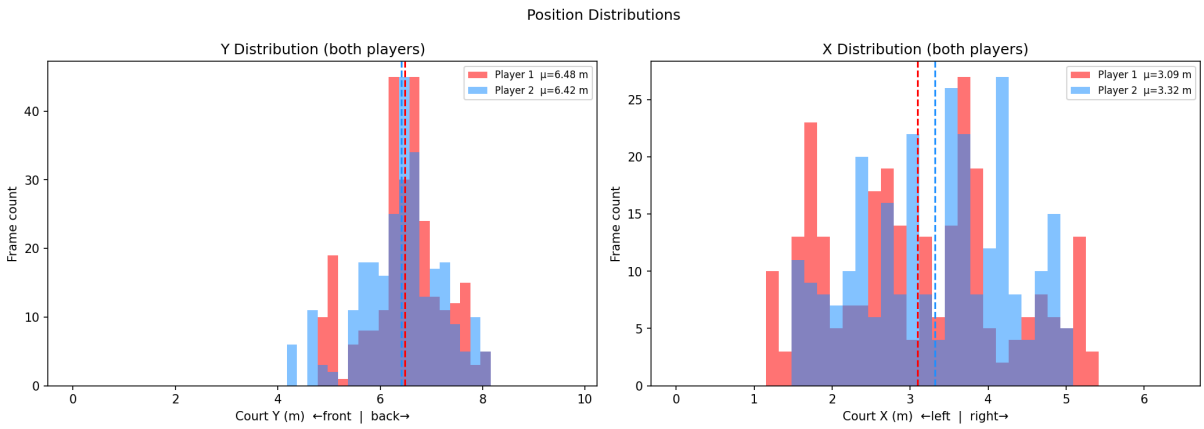


Figure 6: Marginal position distributions for both players. *Left:* court depth (Y axis); both players cluster around 6–7 m (back-court), consistent with the zone breakdown. *Right:* lateral position (X axis); both players centre around 3 m (court midline), as expected.

Testing

A `pytest` test suite covers the major pipeline components:

Test file	What it validates
`test_tracking.py`	Player tracking frame-to-frame consistency
`test_homography.py`	Calibration point reprojection accuracy
`test_stats.py`	Distance/speed computation correctness
`test_zones.py`	Zone assignment logic
`test_timeseries.py`	Time-series analytics
`test_plots.py`	Visualisation output (file existence, format)
`test_validate.py`	Ground-truth validation helpers

In-Progress Work

Ball Detection

Ball detection uses a two-stage approach:

1. **YOLOv8n (COCO class 32, "sports ball")** runs full-frame inference. COCO training data does not include squash balls (black, ∼40 mm diameter), so recall is low.
2. **MOG2 background subtraction** detects foreground blobs, which are filtered by circularity and size to identify ball candidates. This catches fast-moving balls that YOLO misses.

When both methods fire, YOLO is preferred if its detection falls within 60 pixels of the MOG2 candidate; otherwise MOG2 is used.

Current recall estimate: ∼60–70%. The target is ≥70% on a labelled test set (not yet assembled). At 25 fps, a hard-driven ball (∼150 km/h) travels ∼1.67 m per frame, making any frame-skip approach non-viable.

Ball Tracking (Kalman Filter)

A 4-state Kalman filter (x, y, v_x, v_y) smooths the raw detection sequence and fills gaps up to 5 consecutive missed frames using constant-velocity prediction. Gaps longer than 5 frames are logged as "ball lost" segments. The smoothed output is saved separately from the raw detections.

Figure 7 shows the current state of ball tracking on the analysed clip. Only 3 detections were successfully located, illustrating the severity of the ball detection problem.

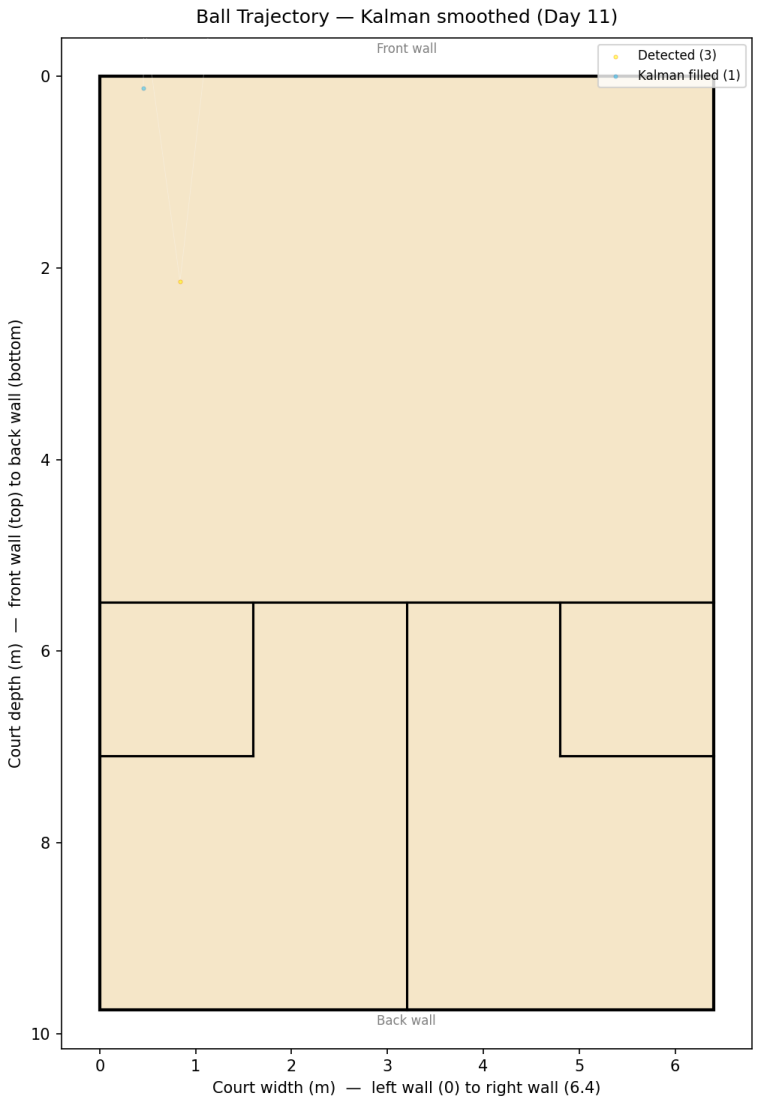


Figure 7: Kalman-smoothed ball trajectory for the analysed clip. Yellow dots are YOLO/MOG2 detections; the blue dot is a Kalman-filled position. Only 3 positions were detected across the entire clip, demonstrating that ball tracking is the primary current blocker.

TrackNetV4 (Experimental)

A custom implementation of the TrackNetV4 architecture has been trained as an alternative ball detector. The model is a U-Net with a motion prompt layer. Inputs are 11 channels: 3 consecutive RGB frames plus 2 inter-frame difference maps. The output is a 3-channel heatmap (one Gaussian blob per input frame indicating the ball position).

Status: The model has been trained on labelled frames from the match video, but inference quality is poor. Ball detections are noisy and unreliable. This remains an active area of investigation.

Rally Segmentation

Rally boundaries are detected by finding gaps in the ball-detection sequence longer than 20 frames (∼0.8 s at 25 fps). For each rally, the following are computed:

- Start and end frame numbers
- Duration in seconds
- Approximate shot count (velocity direction reversals in ball trajectory)
- Per-player distance covered during the rally

Because rally segmentation depends on ball detection, the output is unreliable at current detection recall levels. Figure 8 shows the rally timeline detected from the current clip, and Figure 9 shows both player trajectories and ball detections coloured by rally.

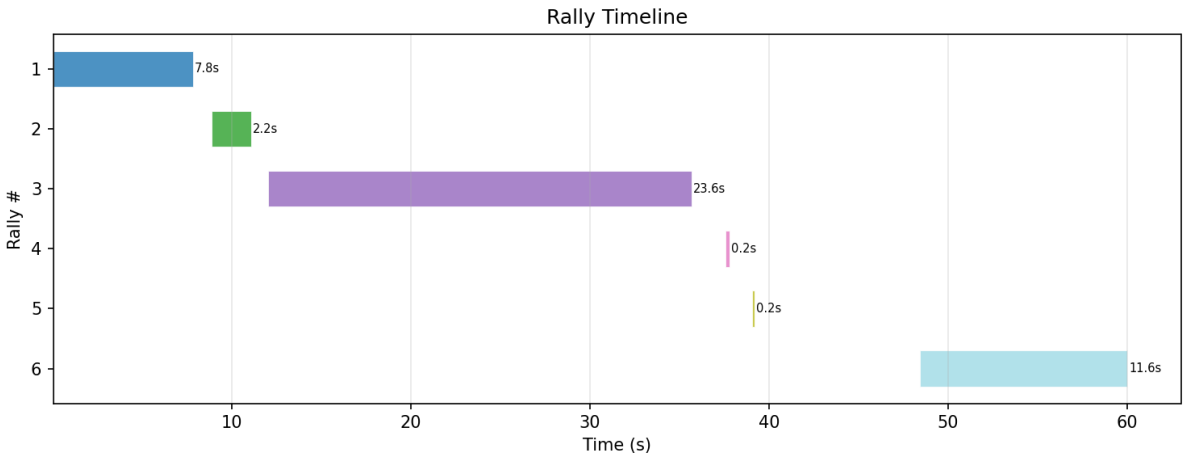


Figure 8: Detected rally timeline. Six rallies were segmented, ranging from 0.2 s to 23.6 s. The very short rallies (0.2 s) are likely artefacts of the low ball detection recall rather than real rallies.

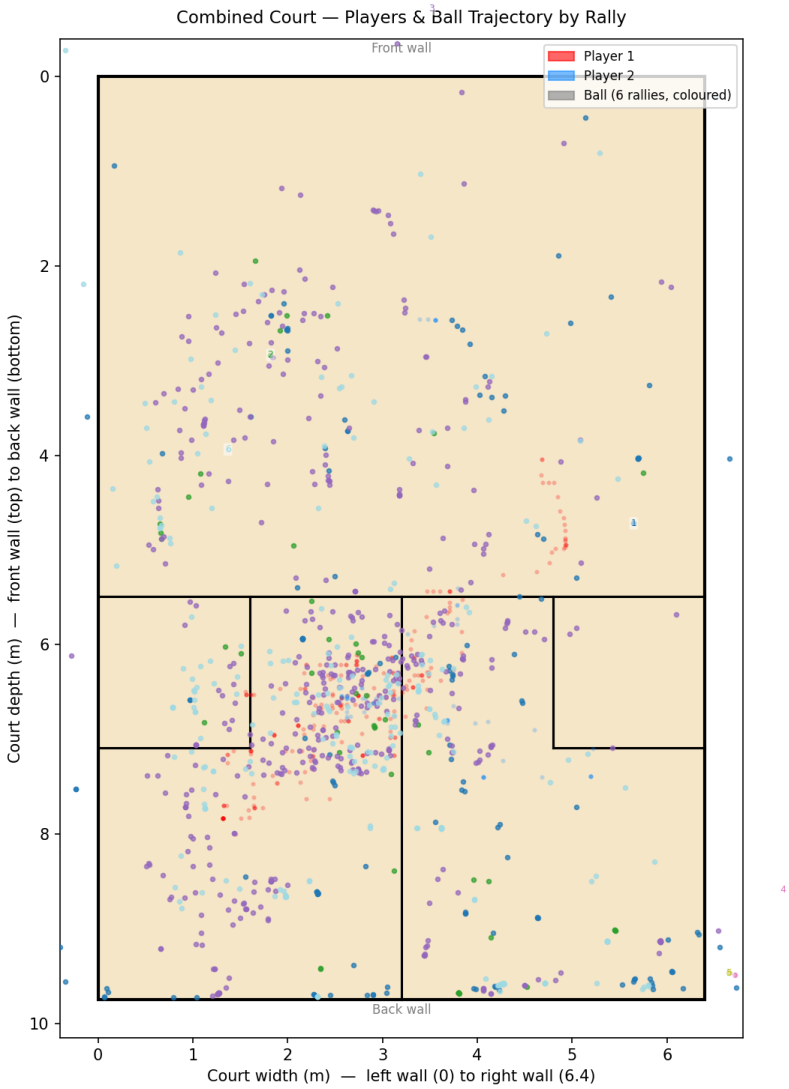


Figure 9: Combined court diagram showing Player 1 (red), Player 2 (blue), and ball detections (coloured by rally). The sparse ball detections across the court highlight the detection gap that needs to be resolved.

Known Issues & Blockers

Issue	Description	Severity
Ball detection recall	∼60–70%, target ≥70%. Squash balls absent from COCO training data. TrackNetV4 trained but not working reliably.	**High**
Player initialisation	If both players are in the same half of the court on frame 1, P1/P2 assignment is incorrect for the entire clip.	**Medium**
Identity swap	No automatic recovery from identity swaps (proximity warning exists but no correction).	**Medium**
Per-rally stats	Stats are computed per-clip; per-rally breakdowns are planned but not yet implemented.	Low
Manual calibration	14-point click UI takes 3–5 minutes per court setup; error-prone on low-resolution footage.	Low
Stats persistence	Match history stored as a flat JSON file; SQLite migration planned but not done.	Low

Remaining Planned Work

Week 3 — Web Interface

- Streamlit dashboard for video upload and results display
- In-browser court calibration (click on video frame)
- PDF report export
- Match history and player profiles

Week 4 — Ball Tracking & Rally Intelligence

- Resolve ball detection recall (≥70% on labelled test set)
- Assemble and label ball detection ground truth dataset
- Fine-tune or replace detection approach (TrackNetV4 or domain-specific model)
- Shot classification (serve, drive, drop, boast, lob) from ball trajectory
- Front wall heatmap
- Automatic video highlight clips

Week 5 — Shot Outcomes & Deployment

- Shot outcome mapping (winner, error, let)
- Momentum analysis (rolling rally win rates)
- End-to-end accuracy report (player position RMSE, ball F1, rally F1)
- Docker container for portable deployment
- Physics-based ball model (UKF with gravity/drag)

Technical Stack

Library	Purpose
OpenCV	Video I/O, background subtraction, homography
Ultralytics (YOLOv8)	Person detection, pose estimation, sports-ball detection
MediaPipe	Legacy pose estimation (kept for comparison)
NumPy / SciPy	Numerical processing, Hungarian assignment, Kalman filter
PyTorch	TrackNetV4 model training and inference
Matplotlib	All visualisations
pytest	Automated testing

Summary

The first two weeks of development have produced a functional player-tracking and statistics pipeline. Court calibration, YOLOv8-pose based tracking, movement metrics, zone breakdowns, and heatmap visualisations all work reliably and are covered by automated tests.

The main current challenge is ball tracking. The squash ball is small, fast, dark, and not well-represented in standard computer vision datasets. Two approaches have been attempted (YOLO+MOG2 and a custom TrackNetV4 implementation); neither yet meets the quality threshold needed to unlock the downstream rally and shot analysis features.

Once ball detection is resolved, the path to a complete match analysis report — covering player movement, rally-by-rally statistics, shot classification, and front wall tendencies — is straightforward.